

07-Python Components and Basic Data Types

Introduction

- The session begins with an overview of Python programming basics, using Google Colab (similar to Anaconda); Colab saves work in Google Drive and offers additional packages.

Python Components

Understanding these building blocks is emphasized as essential for learning any programming language.

Literals

- **Definition:** Any independent value or object (not in list/tuple) in a program; e.g. numbers, strings.
- **Types:**
 - **Integer literals:** Whole numbers, e.g., 10, -10, 0
 - **Float literals:** Numbers with decimals, e.g., 10.5, -2.3
 - **Boolean literals:** `True`, `False`
 - **Complex literals:** Numbers in `a + bj` format, e.g., 10+20j
 - **String literals:** Values inside single, double, or triple quotes, e.g., 'hello', "world", "text"
- **No list/tuple literals:** Lists/tuples are constructed from the above types.

Identifiers

- **Definition:** Any name used in a program (variables, functions, classes).
- **Rules:**
 - Allowed characters: A-Z, a-z, 0-9, _ (underscore)
 - Cannot start with a number, have spaces, or use special characters/hyphens
 - Case-sensitive
 - Reserved words not allowed as identifiers
 - Use consistent naming conventions (e.g., PascalCase for classes)
 - Do not use names starting and ending with double underscores (reserved for Python internals)
 - Constants should be ALL_CAPS

Reserved Words (Keywords)

- Python keywords are pre-booked and cannot be used as identifiers, e.g., `for`, `if`, `def`.
- Examples: `async`, `break`, `class`, `None`.
- Using a keyword as a name causes a syntax error.

Constants

- Defined with ALL_CAPS to distinguish from variables.
- Python does not enforce immutability; this is a convention for clarity.

Expressions & Statements

- **Expression**
 - : Combination of literals, variables, operators that returns a value (can be evaluated in print).
- **Statement**
 - : Instructions that perform actions but do not return a value (e.g., assignment, control flow).
- Expressions yield output; statements do not.

Blocks & Indentation

- **Block**: Code executed as one unit (e.g., inside a loop/if statement)
- **Indentation**: Required in Python to define block structure (replaces `{ }` of other languages)
- Code at same indentation level is in the same block; inconsistent indentation causes errors.
- Colons (`:`) after block-initiating statements (e.g., `if`, `while`, `def`) signal block start.

Comments

- **Single-line**: Precede with `#`. Ignored by Python.
- **Multi-line**: Can use triple quotes (`""" ... """`) for convenience, but this creates an unused string, not a true comment.

- Use comments for code clarification/debugging.

Python Data Types

Python data types are divided into fundamental and derived types. The focus here is fundamentals:

Integer

- Whole numbers without decimals. Default representation is decimal (base 10).
- Can also be expressed in:
 - **Binary:** Base 2 (prefix `0b` or `0B`)
 - **Octal:** Base 8 (prefix `0o` or `0O`)
 - **Hexadecimal:** Base 16 (prefix `0x` or `0X`)
- Conversion functions: `bin()`, `oct()`, `hex()` convert to appropriate representations.
- Only integers can use these binary/octet/hex representations.
- Integers have dynamic memory allocation (no 'long' or 'short'). Size grows with value.

Float

- Numbers with decimal points, e.g., 3.14.
- Only use decimal representation.

- Some type conversions allowed (`float('10.5')`), but not from complex or invalid strings.

Complex

- Format: `a + bj` (e.g., `2 + 3j`), with real part `a` and imaginary part `b`.
- Use `complex(real, imag)` constructor. If the first argument is a string, cannot pass a second argument.
- Retrieve real/imaginary parts with `.real`, `.imag` properties.

Boolean

- Only two values: `True` (1), `False` (0).
- Type conversion: Zero/empty values return `False`, any other value returns `True`.
- Applies to numbers, strings, collections, etc.

None

- Represents missing or undefined values.

Input Handling & Type Conversion

- Input from the user is always a string; must be converted to appropriate types (`int(input())`, `float(input())`, `complex(input(), input())`).

- Invalid conversions trigger errors (e.g., entering a string when an integer is expected).
- Use `eval()` in combination with `input()` for flexible input parsing.

Practice Recommendations & Resources

- Regular Python practice is strongly recommended (at least 1 hour daily), regardless of background.
- Online resource: w3resource.com for exercises and solutions
- Engage with assignment questions and not simply copy code; sample test/exercise sites suggested.
- Practice is key to retaining concepts and avoiding simple mistakes in interviews or assessments.

Closing Notes

- The lecture provides assignments for reinforcing concepts.
- All main lessons: importance of understanding core components (syntax, naming, keywords, blocks, data types, type conversion, input, and commenting).

Notes powered by [CraftNote](#)